

# Searching and Sorting Algorithms

A Complete Reference — Definitions, Examples, Code, and Complexity Proofs

Data Structures Reference Notes

September 2026

## Contents

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                             | <b>2</b>  |
| 1.1      | What is Searching? . . . . .                    | 2         |
| 1.2      | What is Sorting? . . . . .                      | 2         |
| 1.3      | Why They Matter . . . . .                       | 2         |
| 1.4      | A Quick Primer on Complexity Notation . . . . . | 2         |
| <b>2</b> | <b>Searching Algorithms</b>                     | <b>3</b>  |
| 2.1      | Linear Search . . . . .                         | 3         |
| 2.2      | Binary Search . . . . .                         | 5         |
| <b>3</b> | <b>Sorting Algorithms</b>                       | <b>7</b>  |
| 3.1      | Bubble Sort . . . . .                           | 7         |
| 3.2      | Selection Sort . . . . .                        | 9         |
| 3.3      | Insertion Sort . . . . .                        | 11        |
| 3.4      | Merge Sort . . . . .                            | 13        |
| 3.5      | Quick Sort . . . . .                            | 15        |
| 3.6      | Counting Sort . . . . .                         | 17        |
| 3.7      | Radix Sort . . . . .                            | 19        |
| <b>4</b> | <b>Summary Table</b>                            | <b>22</b> |

# 1 Introduction

## 1.1 What is Searching?

**Searching** is the process of finding whether a particular value (called the **key**) exists within a collection of data, and if so, locating its position. Every application that stores data — a phone book, a database, a file system — needs an efficient way to answer the question: “*is this value present, and where?*”

## 1.2 What is Sorting?

**Sorting** is the process of rearranging a collection of elements into a specific order — usually ascending or descending. Sorting is rarely the end goal by itself; it is almost always a means to make some *other* operation cheaper. A sorted array allows binary search (fast lookup), efficient merging, efficient duplicate detection, and range queries — none of which are efficient on unsorted data.

## 1.3 Why They Matter

Searching and sorting appear everywhere: bank records, airport traffic systems, search engines, leaderboards, file systems, databases, and more. The central trade-off that runs through this entire document is:

**Sorting costs time upfront, but pays for itself through cheaper searches, merges, and lookups later.**

Choosing the right algorithm for the right situation — the size of the data, whether it is nearly sorted, how large the value range is, whether extra memory is available — is the core skill this document builds toward.

## 1.4 A Quick Primer on Complexity Notation

Before analyzing any algorithm, we need a precise language for describing “how fast” it is, independent of the specific computer running it.

- **$O(g(n))$**  — *Big-O*, an **upper bound**. “The algorithm never takes more than roughly  $g(n)$  time” (for large  $n$ ).
- **$\Omega(g(n))$**  — *Big-Omega*, a **lower bound**. “The algorithm never takes less than roughly  $g(n)$  time.”
- **$\Theta(g(n))$**  — *Big-Theta*, a **tight bound**. Both of the above hold simultaneously — the algorithm’s running time grows *exactly* like  $g(n)$ .

Formally,  $f(n) = O(g(n))$  means there exist constants  $c > 0$  and  $n_0$  such that  $0 \leq f(n) \leq c \cdot g(n)$  for all  $n \geq n_0$ . The other two definitions are the mirror-image (for  $\Omega$ ) and the combination (for  $\Theta$ ) of this idea.

We also distinguish three cases for any algorithm:

- **Best case** — the most favorable input (e.g., already-sorted data).
- **Worst case** — the least favorable input (e.g., reverse-sorted data). This is the guarantee you can always rely on.
- **Average case** — expected performance over random/typical input.

## 2 Searching Algorithms

Throughout this document we use one running example array:

$$A = [8, 3, 5, 1, 9, 2]$$

Its sorted form — used wherever a *sorted* array is required (binary search, and later as the goal of every sorting algorithm) — is:

$$A_{sorted} = [1, 2, 3, 5, 8, 9]$$

### 2.1 Linear Search

#### 2.1.1 Definition

Linear search checks every element of an array **one by one**, in order, comparing it against the key, until a match is found or the array is exhausted. It requires **no assumption** about the order of the data — it works on sorted or unsorted arrays equally.

#### 2.1.2 How It Works — Example

Search for **key = 9** in  $A = [8, 3, 5, 1, 9, 2]$  (unsorted — linear search does not care):

| Step | Index checked | Value | Match?     |
|------|---------------|-------|------------|
| 1    | 0             | 8     | No         |
| 2    | 1             | 3     | No         |
| 3    | 2             | 5     | No         |
| 4    | 3             | 1     | No         |
| 5    | 4             | 9     | <b>Yes</b> |

Found at index 4, after 5 comparisons.

#### 2.1.3 Code

```
int linearSearch(int arr[], int n, int key) {  
    for (int i = 0; i < n; i++) {  
        if (arr[i] == key) {  
            return i;          // found -- return index  
        }  
    }  
    return -1;                 // not found  
}
```

### 2.1.4 Code Walkthrough (same example, line by line)

Call: `linearSearch(A, 6, 9)`

| Iteration | i | arr[i] | arr[i] == key?     | Action           |
|-----------|---|--------|--------------------|------------------|
| 1         | 0 | 8      | 8==9 → false       | loop continues   |
| 2         | 1 | 3      | 3==9 → false       | loop continues   |
| 3         | 2 | 5      | 5==9 → false       | loop continues   |
| 4         | 3 | 1      | 1==9 → false       | loop continues   |
| 5         | 4 | 9      | 9==9 → <b>true</b> | <b>return 4;</b> |

The function returns **4**.

### 2.1.5 Time Complexity

- **Best case:  $O(1)$ .** The key is the very first element checked — one comparison.
- **Worst case:  $O(n)$ .** The key is the last element, or absent entirely — every one of the  $n$  elements must be checked.
- **Average case:  $O(n)$ .** For a key at a random position, the expected number of comparisons is about  $n/2$ , which is still  $\Theta(n)$ .

**Intuitive proof of the worst case ( $\Omega(n)$ ):** Consider an adversary who places the key at the very last position (or removes it entirely). Because the algorithm has no information about where the key is, and no ordering to exploit, it cannot skip checking any earlier index — skipping one could mean missing the key. Therefore every algorithm that solves this problem correctly must, in the worst case, examine all  $n$  elements. Combined with the  $O(n)$  upper bound from simply counting loop iterations, this gives  $\Theta(n)$  for the worst case.

## 2.2 Binary Search

### 2.2.1 Definition

Binary search finds a key in a **sorted** array by repeatedly comparing the key to the middle element of the current search range, and eliminating **half** of the remaining range based on that comparison. It requires the array to be sorted beforehand.

### 2.2.2 How It Works — Example

Search for **key = 8** in  $A_{sorted} = [1, 2, 3, 5, 8, 9]$  (indices 0 to 5):

| Step | first | last | mid | A[mid] | Comparison | Action                       |
|------|-------|------|-----|--------|------------|------------------------------|
| 1    | 0     | 5    | 2   | 3      | $8 > 3$    | search right half: first = 3 |
| 2    | 3     | 5    | 4   | 8      | $8 == 8$   | <b>found at index 4</b>      |

Found in just 2 comparisons, versus up to 6 for linear search on the same array.

### 2.2.3 Code

```
int binarySearch(int arr[], int n, int key) {
    int first = 0, last = n - 1;

    while (first <= last) {
        int mid = (first + last) / 2;

        if (arr[mid] == key) {
            return mid;           // found
        } else if (key < arr[mid]) {
            last = mid - 1;       // search left half
        } else {
            first = mid + 1;      // search right half
        }
    }
    return -1;                   // not found
}
```

### 2.2.4 Code Walkthrough (same example, line by line)

Call: `binarySearch(A_sorted, 6, 8)`. Initial: first = 0, last = 5.

| Iteration | first | last | mid =<br>(first+last)/2 | arr[mid] | Comparison                       | Update                  |
|-----------|-------|------|-------------------------|----------|----------------------------------|-------------------------|
| 1         | 0     | 5    | 2                       | 3        | $8 < 3$ ? No<br>→ else<br>branch | first =<br>mid+1 =<br>3 |

| Iteration | first | last | mid =<br>(first+last)/2 | arr[mid] | Comparison                | Update       |
|-----------|-------|------|-------------------------|----------|---------------------------|--------------|
| 2         | 3     | 5    | 4                       | 8        | arr[mid] == key<br>→ true | return<br>4; |

The function returns **4**. Loop condition `first <= last` never even needs to fail here — the match is found directly on iteration 2.

### 2.2.5 Time Complexity

- **Best case:  $O(1)$ .** The key is exactly at the first computed middle.
- **Worst case:  $\Theta(\log n)$ .** The key is absent, or found only after the search window collapses to size 1.
- **Average case:  $\Theta(\log n)$ .**

**Intuitive proof:** Let  $T(n)$  be the worst-case number of comparisons on an array of size  $n$ . Each call does  $O(1)$  work (one comparison at `mid`) and then recurses on a window of size **at most**  $n/2$  — never both halves, only one. This gives the recurrence:

$$T(n) = T(n/2) + O(1)$$

Unroll it repeatedly:

$$T(n) = T(n/2) + c = T(n/4) + 2c = T(n/8) + 3c = \dots = T(n/2^k) + kc$$

This bottoms out when  $n/2^k = 1$ , i.e.  $k = \log_2 n$ . At that point  $T(1) = O(1)$ , so:

$$T(n) = O(1) + c \log_2 n = \Theta(\log n)$$

**Why it cannot be faster than  $\Omega(\log n)$ :** each comparison against `arr[mid]` provides only enough information to discard *at most* half of the remaining candidates — you cannot legally throw away more than half while still guaranteeing correctness on a sorted array, since anything within the surviving half is still a legitimate candidate. So the number of comparisons needed is exactly “how many times can  $n$  be halved before reaching 1,” which is  $\log_2 n$  by definition. Combining the upper and lower bound gives the tight bound  $\Theta(\log n)$ .

## 3 Sorting Algorithms

We now sort the same array throughout:

$$A = [8, 3, 5, 1, 9, 2] \longrightarrow [1, 2, 3, 5, 8, 9]$$

### 3.1 Bubble Sort

#### 3.1.1 Definition

Bubble sort repeatedly steps through the array, comparing **adjacent** pairs of elements and swapping them if they are in the wrong order. Each full pass “bubbles” the largest unsorted element to its correct position at the end.

#### 3.1.2 How It Works — Example

Pass 1 on  $A = [8, 3, 5, 1, 9, 2]$ :

| Compare | Result       | Array after        |
|---------|--------------|--------------------|
| (8,3)   | 8>3, swap    | [3, 8, 5, 1, 9, 2] |
| (8,5)   | 8>5, swap    | [3, 5, 8, 1, 9, 2] |
| (8,1)   | 8>1, swap    | [3, 5, 1, 8, 9, 2] |
| (8,9)   | 8<9, no swap | [3, 5, 1, 8, 9, 2] |
| (9,2)   | 9>2, swap    | [3, 5, 1, 8, 2, 9] |

End of pass 1: **[3, 5, 1, 8, 2, 9]** — the largest element (9) is now correctly at the end.

Continuing all passes eventually yields: **[1, 2, 3, 5, 8, 9]**.

#### 3.1.3 Code

```
void bubbleSort(int arr[], int n) {
    for (int k = 0; k < n - 1; k++) {
        int swapped = 0;
        for (int j = 0; j < n - k - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }
        if (!swapped) break; // already sorted -- stop early
    }
}
```

### 3.1.4 Code Walkthrough (same example, line by line — Pass 1 only, k=0)

$n=6$ , so inner loop runs  $j = 0$  to  $4$  ( $n-k-1 = 5$ ).

| $j$ | $arr[j]$ | $arr[j+1]$ | $arr[j] > arr[j+1]?$ | Swap? | swapped | Array state   |
|-----|----------|------------|----------------------|-------|---------|---------------|
| 0   | 8        | 3          | Yes                  | Yes   | 1       | [3,8,5,1,9,2] |
| 1   | 8        | 5          | Yes                  | Yes   | 1       | [3,5,8,1,9,2] |
| 2   | 8        | 1          | Yes                  | Yes   | 1       | [3,5,1,8,9,2] |
| 3   | 8        | 9          | No                   | No    | 1       | [3,5,1,8,9,2] |
| 4   | 9        | 2          | Yes                  | Yes   | 1       | [3,5,1,8,2,9] |

swapped=1 at the end, so  $k=1$  (pass 2) begins. This repeats until a pass makes no swaps.

### 3.1.5 Time Complexity

- **Worst / Average case:**  $O(n^2)$ .
- **Best case:**  $O(n)$  — only with the swapped early-exit flag, on an already-sorted array.

**Intuitive proof (worst case):** The outer loop runs  $n - 1$  times. For each outer iteration  $k$ , the inner loop runs  $n - k - 1$  times. Summing over all  $k$ :

$$\sum_{k=0}^{n-2} (n - k - 1) = (n - 1) + (n - 2) + \cdots + 1 = \frac{n(n - 1)}{2} = O(n^2)$$

This total is **fixed by the loop structure alone** — every pair in this triangular pattern is compared regardless of input, so the worst case (and, without the early-exit flag, every case) is  $\Theta(n^2)$ .

**Intuitive proof (best case, with early exit):** If the array is already sorted, the very first pass makes zero swaps ( $arr[j] > arr[j+1]$  is false for every adjacent pair). The swapped flag stays 0, and the algorithm breaks out after just one pass of  $n - 1$  comparisons — giving  $O(n)$ .

## 3.2 Selection Sort

### 3.2.1 Definition

Selection sort repeatedly finds the **minimum** element of the remaining unsorted portion and swaps it into the front of that portion. Unlike bubble sort, it performs exactly one swap per pass.

### 3.2.2 How It Works — Example

Pass 1 on  $A = [8, 3, 5, 1, 9, 2]$ : scan the whole array, find the minimum.

Minimum is **1**, at index 3. Swap `arr[0]` and `arr[3]`:

$$[8, 3, 5, 1, 9, 2] \rightarrow [1, 3, 5, 8, 9, 2]$$

Pass 2: scan indices 1..5, minimum is **2** at index 5. Swap `arr[1]` and `arr[5]`:

$$[1, 3, 5, 8, 9, 2] \rightarrow [1, 2, 5, 8, 9, 3]$$

Continuing this process for all remaining passes yields: **[1, 2, 3, 5, 8, 9]**.

### 3.2.3 Code

```
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        if (minIndex != i) {
            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
    }
}
```

### 3.2.4 Code Walkthrough (same example, line by line — Pass 1 only, i=0)

`minIndex` starts at 0 (`arr[0]=8`). Inner loop scans `j = 1` to 5:

| j | arr[j] | arr[j] < arr[minIndex]? | minIndex after |
|---|--------|-------------------------|----------------|
| 1 | 3      | 3<8 → true              | 1              |
| 2 | 5      | 5<3 → false             | 1              |
| 3 | 1      | 1<3 → true              | 3              |
| 4 | 9      | 9<1 → false             | 3              |

| j | arr[j] | arr[j] < arr[minIndex]? | minIndex after |
|---|--------|-------------------------|----------------|
| 5 | 2      | 2 < 1 → false           | 3              |

Inner loop ends with `minIndex = 3`. Since `3 != 0`, swap `arr[0]` and `arr[3]`:

`[8, 3, 5, 1, 9, 2] → [1, 3, 5, 8, 9, 2]`

### 3.2.5 Time Complexity

- **Best, average, and worst case: all  $\Theta(n^2)$ .**

**Intuitive proof:** The outer loop runs  $n - 1$  times, and for each  $i$ , the inner loop **must** run its full  $n - i - 1$  comparisons — there is no shortcut, because the only way to be certain you’ve found the true minimum of a range is to compare it against every element in that range. Unlike bubble sort, there is no condition here that can trigger an early stop; finding a minimum always requires a full scan. So:

$$\sum_{i=0}^{n-2} (n - i - 1) = \frac{n(n-1)}{2} = \Theta(n^2)$$

for **every input**, with no best-case improvement possible. Selection sort’s only advantage is that it performs exactly  $n - 1$  swaps total, regardless of input — the fewest writes of any comparison sort covered here.

## 3.3 Insertion Sort

### 3.3.1 Definition

Insertion sort builds a sorted region one element at a time. At each step, it takes the next unsorted element (the **key**) and shifts elements in the sorted region rightward until it finds the correct gap to insert the key.

### 3.3.2 How It Works — Example

Start: [8] is trivially sorted. Insert **3**:

$3 < 8$ , shift 8 right, insert 3  $\rightarrow$  [3, 8, 5, 1, 9, 2]

Insert **5**: sorted prefix is [3, 8].  $5 < 8$ , shift 8 right;  $5 > 3$ , stop, insert  $\rightarrow$  [3, 5, 8, 1, 9, 2]

Insert **1**: sorted prefix is [3, 5, 8]. 1 is smaller than all three — shift all three right, insert at front  $\rightarrow$  [1, 3, 5, 8, 9, 2]

Continuing similarly for 9 and 2 yields: [1, 2, 3, 5, 8, 9].

### 3.3.3 Code

```
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];    // shift right
            j--;
        }
        arr[j + 1] = key;          // insert key in the gap
    }
}
```

### 3.3.4 Code Walkthrough (same example, line by line — inserting `arr[3]=1`, i.e. `i=3`)

Before this step: `arr = [3, 5, 8, 1, 9, 2]`. `key = arr[3] = 1`, `j = 2`.

| Check | j  | arr[j] | arr[j] > key?                               | Action                                          | Array state        |
|-------|----|--------|---------------------------------------------|-------------------------------------------------|--------------------|
| 1     | 2  | 8      | $8 > 1 \rightarrow \text{true}$             | <code>arr[3]=arr[2]</code><br><code>j=1</code>  | [3, 8, 8, 9, 2]    |
| 2     | 1  | 5      | $5 > 1 \rightarrow \text{true}$             | <code>arr[2]=arr[1]</code><br><code>j=0</code>  | [3, 5, 8, 9, 2]    |
| 3     | 0  | 3      | $3 > 1 \rightarrow \text{true}$             | <code>arr[1]=arr[0]</code><br><code>j=-1</code> | [3, 3, 5, 8, 9, 2] |
| 4     | -1 | —      | <code>j &gt;= 0</code> fails,<br>loop stops | <code>arr[j+1]=arr[0]</code>                    | [1, 3, 5, 8, 9, 2] |

### 3.3.5 Time Complexity

- **Worst case:**  $O(n^2)$  — reverse-sorted input; every new key must shift past *all* previously sorted elements.
- **Best case:**  $O(n)$  — already-sorted input; the `while` condition is false immediately for every `i`, so zero shifts occur, only  $n - 1$  comparisons total.
- **Average case:**  $O(n^2)$ .

**Intuitive proof (worst case):** On reverse-sorted input, inserting the element at position  $i$  requires shifting all  $i$  already-sorted elements. Summing over all insertions:

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = O(n^2)$$

**Intuitive proof (best case):** If the array is already sorted, then for every  $i$ ,  $\text{key} = \text{arr}[i] \geq \text{arr}[i-1]$ , so the condition `arr[j] > key` is false on the very first check — the `while` loop body never executes. Total work is  $n - 1$  outer iterations  $\times O(1)$  each  $= O(n)$ .

## 3.4 Merge Sort

### 3.4.1 Definition

Merge sort is a **divide-and-conquer** algorithm: split the array into two halves, recursively sort each half, then **merge** the two sorted halves back into one sorted array. It guarantees  $O(n \log n)$  performance regardless of input.

### 3.4.2 How It Works — Example

**Divide**  $A = [8, 3, 5, 1, 9, 2]$  down to single elements:

$$[8, 3, 5, 1, 9, 2] \rightarrow [8, 3, 5] [1, 9, 2] \rightarrow [8] [3, 5] [1, 9] [2]$$

**Merge** back up, level by level:

- Merge  $[3]$  and  $[5]$  (from splitting  $[3, 5]$ )  $\rightarrow [3, 5]$ ; merge with  $[8]$   $\rightarrow [3, 5, 8]$
- Merge  $[1]$  and  $[9]$   $\rightarrow [1, 9]$ ; merge with  $[2]$   $\rightarrow [1, 2, 9]$
- Final merge of  $[3, 5, 8]$  and  $[1, 2, 9]$ , comparing fronts at each step:

| Compare fronts   | Take | Result so far |
|------------------|------|---------------|
| 3 vs 1           | 1    | [1]           |
| 3 vs 2           | 2    | [1,2]         |
| 3 vs 9           | 3    | [1,2,3]       |
| 5 vs 9           | 5    | [1,2,3,5]     |
| 8 vs 9           | 8    | [1,2,3,5,8]   |
| (left exhausted) | 9    | [1,2,3,5,8,9] |

**Final:**  $[1, 2, 3, 5, 8, 9]$

### 3.4.3 Code

```
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}
```

```

}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

```

### 3.4.4 Code Walkthrough (same example, line by line — the final merge: L=[3, 5, 8], R=[1, 2, 9])

n1=3, n2=3. Initial: i=0, j=0, k=0.

| Step | Compare          | L[i] <= R[j]? | Action                  | i | j | k |
|------|------------------|---------------|-------------------------|---|---|---|
| 1    | L[0]=3 vs R[0]=1 | No            | arr[0]=R[0]=1, j++, k++ | 0 | 1 | 1 |
| 2    | L[0]=3 vs R[1]=2 | No            | arr[1]=R[1]=2, j++, k++ | 0 | 2 | 2 |
| 3    | L[0]=3 vs R[2]=9 | Yes           | arr[2]=L[0]=3, i++, k++ | 1 | 2 | 3 |
| 4    | L[1]=5 vs R[2]=9 | Yes           | arr[3]=L[1]=5, i++, k++ | 2 | 2 | 4 |
| 5    | L[2]=8 vs R[2]=9 | Yes           | arr[4]=L[2]=8, i++, k++ | 3 | 2 | 5 |

Now  $i < n1$  fails ( $3 < 3$  false) — left leftover loop:  $\text{arr}[5] = \text{R}[2] = 9$ .

**Result:** **arr** = [1, 2, 3, 5, 8, 9] (correct)

### 3.4.5 Time Complexity

- **Best, average, and worst case:** all  $\Theta(n \log n)$ .

**Intuitive proof:** Each call splits the array ( $O(1)$  work) into two halves of size  $n/2$ , and merges the results in  $O(n)$  time (a single linear pass, since both halves are already sorted). This gives the recurrence:

$$T(n) = 2T(n/2) + O(n)$$

Picture this as a recursion tree. At the root (level 0), the merge cost is  $n$ . At level 1, there are 2 sub-problems of size  $n/2$  each, so total merge cost is  $2 \times (n/2) = n$ . At level 2, there are 4 sub-problems of size  $n/4$ , so total merge cost is  $4 \times (n/4) = n$ . **Every level costs exactly  $n$** , regardless of how many pieces it is divided into — because doubling the pieces always exactly cancels halving their size.

The number of levels is determined by “how many times can  $n$  be halved before reaching 1,” which is  $\log_2 n$ . Multiplying the cost-per-level by the number of levels:

$$T(n) = n \times \log_2 n = \Theta(n \log n)$$

This holds for **every input** — no case is worse than another, because the split is always exactly by index (never by comparing values), and the merge always does a full linear pass regardless of the data’s original order.

## 3.5 Quick Sort

### 3.5.1 Definition

Quicksort is also divide-and-conquer, but does its hard work **before** recursing: it picks a **pivot** element, **partitions** the array so everything smaller than the pivot is on its left and everything larger is on its right (placing the pivot in its final sorted position), then recursively sorts each side independently. No merge step is needed.

### 3.5.2 How It Works — Example

On  $A = [8, 3, 5, 1, 9, 2]$ , using the **last element as pivot** (Lomuto scheme):

Pivot = **2** (last element). Scan  $j=0$  to 4, tracking boundary  $i$  (elements  $\leq$  pivot so far):

| $j$ | $arr[j]$ | $\leq \text{pivot}(2)$ ? | Action                                                       |
|-----|----------|--------------------------|--------------------------------------------------------------|
| 0   | 8        | No                       | skip                                                         |
| 1   | 3        | No                       | skip                                                         |
| 2   | 5        | No                       | skip                                                         |
| 3   | 1        | Yes                      | $i=0$ , swap $arr[0], arr[3] \rightarrow [1, 3, 5, 8, 9, 2]$ |
| 4   | 9        | No                       | skip                                                         |

Swap pivot into position  $i+1=1$ : swap  $arr[1]$  and  $arr[5] \rightarrow [1, 2, 5, 8, 9, 3]$

Pivot 2 is now fixed at index 1. Recurse on left  $[1]$  (already sorted, base case) and right  $[5, 8, 9, 3]$  (new pivot each time, repeating the same process) until fully sorted: **[1, 2, 3, 5, 8, 9]**.

### 3.5.3 Code

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
        }
    }
    int temp = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = temp;
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

```

    }
}

```

### 3.5.4 Code Walkthrough (same example, line by line — first call `partition(arr, 0, 5)`)

`pivot = arr[5] = 2. i = -1.`

| j | arr[j] | arr[j] <= pivot? | Action                                   | i after |
|---|--------|------------------|------------------------------------------|---------|
| 0 | 8      | No               | —                                        | -1      |
| 1 | 3      | No               | —                                        | -1      |
| 2 | 5      | No               | —                                        | -1      |
| 3 | 1      | Yes              | i++, swap arr[0], arr[3] → [1,3,5,8,9,2] | 0       |
| 4 | 9      | No               | —                                        | 0       |

Loop ends. Final swap: `arr[i+1]=arr[1]` with `arr[high]=arr[5]`: swap `arr[1]` and `arr[5]` → **[1, 2, 5, 8, 9, 3]**. `partition` returns `i+1 = 1`.

`quickSort` then recurses: `quickSort(arr, 0, 0)` (base case, nothing to do) and `quickSort(arr, 2, 5)` (recurses further on [5, 8, 9, 3] with a new pivot).

### 3.5.5 Time Complexity

- **Best / average case:**  $\Theta(n \log n)$  — pivot splits the array reasonably evenly each time.
- **Worst case:**  $O(n^2)$  — pivot is always the smallest or largest remaining element (e.g., already-sorted input with last-element pivot).

**Intuitive proof (best/average case):** If the pivot splits the array roughly in half, the recurrence is identical to merge sort's:

$$T(n) = 2T(n/2) + O(n) \implies T(n) = \Theta(n \log n)$$

by the same recursion-tree argument (the  $O(n)$  partition cost plays the same role as merge sort's  $O(n)$  merge cost).

**Intuitive proof (worst case):** If the pivot is always the extreme element, one side of every partition is empty:

$$T(n) = T(n-1) + O(n)$$

Unrolling:  $T(n) = O(n) + O(n-1) + \dots + O(1) = \frac{n(n-1)}{2} = O(n^2)$ . Picture the recursion tree here as a tall, skinny line instead of a short, wide tree —  $n$  levels deep instead of  $\log n$ , because each partition only manages to fix one element's final position instead of roughly halving the remaining work.

**Practical fix:** choosing the pivot **randomly** (or via median-of-three) makes this worst case statistically vanishing rather than eliminating it in theory — no fixed input can reliably trigger it anymore.

## 3.6 Counting Sort

### 3.6.1 Definition

Counting sort is a **non-comparison-based** sort: instead of comparing elements to each other, it counts how many times each distinct value occurs, then reconstructs the sorted array directly from those counts. It requires the value range ( $k$ ) to be known and reasonably small.

### 3.6.2 How It Works — Example

On  $A = [8, 3, 5, 1, 9, 2]$ , values range 1 to 9 ( $k = 9$ ).

**Step 1 — count occurrences:**

|       |   |   |   |   |   |   |   |   |   |
|-------|---|---|---|---|---|---|---|---|---|
| value | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| count | 1 | 1 | 1 | 0 | 1 | 0 | 0 | 1 | 1 |

**Step 2 — reconstruct output** by walking values 1 to 9 in order and outputting each the correct number of times:

[1, 2, 3, 5, 8, 9]

### 3.6.3 Code (stable version)

```
void countingSort(int arr[], int n, int k) {
    int count[k + 1];
    int output[n];

    for (int i = 0; i <= k; i++) count[i] = 0;
    for (int i = 0; i < n; i++) count[arr[i]]++;

    for (int v = 1; v <= k; v++) count[v] += count[v - 1];    // cumulative

    for (int i = n - 1; i >= 0; i--) {                        // backward scan
        output[count[arr[i]] - 1] = arr[i];
        count[arr[i]]--;
    }
    for (int i = 0; i < n; i++) arr[i] = output[i];
}
```

### 3.6.4 Code Walkthrough (same example, line by line)

$n=6$ ,  $k=9$ ,  $arr=[8, 3, 5, 1, 9, 2]$ .

**Tally (Step 2 of code):**  $count = [0, 1, 1, 1, 0, 1, 0, 0, 1, 1]$  (indices 0..9)

**Cumulative sum (Step 3):**  $count = [0, 1, 2, 3, 3, 4, 4, 4, 5, 6]$

**Backward placement (Step 4):** scanning  $i = 5$  down to 0:

| i | arr[i] | count[arr[i]]<br>(before) | Action      | count[arr[i]]<br>(after) |
|---|--------|---------------------------|-------------|--------------------------|
| 5 | 2      | 2                         | output[1]=2 | 1                        |
| 4 | 9      | 6                         | output[5]=9 | 5                        |
| 3 | 1      | 1                         | output[0]=1 | 0                        |
| 2 | 5      | 4                         | output[3]=5 | 3                        |
| 1 | 3      | 3                         | output[2]=3 | 2                        |
| 0 | 8      | 5                         | output[4]=8 | 4                        |

**output** = [1, 2, 3, 5, 8, 9] (correct)

### 3.6.5 Time Complexity

- **All cases (best, average, worst):**  $O(n + k)$ .

**Intuitive proof:** Every loop in the algorithm — initializing `count`, tallying, the cumulative sum, the backward placement, and the final copy — is a **fixed-range** loop with no data-dependent branching that could skip or repeat work. Initialization and the cumulative sum each run in  $O(k)$ ; tallying, placement, and copying each run in  $O(n)$ . Adding these gives  $O(n + k)$ , and since no step's iteration count depends on the *values* in the array (only their count), this bound is identical for every input — there is no better or worse case.

**Why this beats  $\Omega(n \log n)$ :** the comparison-sort lower bound only applies to algorithms that determine order purely by comparing pairs of elements. Counting sort never compares two array elements to each other at all — it only counts — so that lower bound simply does not apply here. The trade-off is the requirement that  $k$  (the value range) be small; if  $k$  grows much larger than  $n$ , the  $O(k)$  terms dominate and the algorithm becomes inefficient.

## 3.7 Radix Sort

### 3.7.1 Definition

Radix sort sorts multi-digit numbers by applying a **stable** counting sort to each digit position, starting from the **least significant digit (LSD)** and moving toward the most significant. Because counting sort is stable, each pass refines the ordering from the previous pass without destroying it.

### 3.7.2 How It Works — Example

For this algorithm we use a separate example with multi-digit numbers, since our running example is all single-digit:

$$B = [91, 19, 55, 54, 37]$$

**Pass 1 — sort by ones digit** (1, 9, 5, 4, 7):

$$[91, 54, 55, 37, 19]$$

(ones digits now in order: 1, 4, 5, 7, 9)

**Pass 2 — sort by tens digit** (9, 5, 5, 3, 1):

$$[19, 37, 54, 55, 91]$$

Note 54 and 55 tie on the tens digit (both 5) — stability preserves their order from pass 1 (54 was already before 55).

**Final: [19, 37, 54, 55, 91]** — fully sorted after 2 passes (2-digit numbers).

### 3.7.3 Code

```
int getMax(int arr[], int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > max) max = arr[i];
    return max;
}

void countingSortByDigit(int arr[], int n, int exp) {
    int output[n];
    int count[10] = {0};

    for (int i = 0; i < n; i++)
        count[(arr[i] / exp) % 10]++;

    for (int i = 1; i < 10; i++)
        count[i] += count[i - 1];
}
```

```

    for (int i = n - 1; i >= 0; i--) {
        int digit = (arr[i] / exp) % 10;
        output[count[digit] - 1] = arr[i];
        count[digit]--;
    }
    for (int i = 0; i < n; i++)
        arr[i] = output[i];
}

void radixSort(int arr[], int n) {
    int max = getMax(arr, n);
    for (int exp = 1; max / exp > 0; exp *= 10) {
        countingSortByDigit(arr, n, exp);
    }
}

```

### 3.7.4 Code Walkthrough (same example, line by line — Pass 1, exp=1, ones digit)

B = [91, 19, 55, 54, 37]. (arr[i] / 1) % 10 extracts the ones digit.

**Tally:** digits are 1, 9, 5, 4, 7 → count[1]=1, count[4]=1, count[5]=1, count[7]=1, count[9]=1 (rest 0)

**Cumulative:** count = [0, 1, 1, 1, 2, 3, 3, 4, 4, 5] (indices 0..9)

**Backward placement,** i=4 down to 0:

| i | arr[i] | ones digit | count[digit] before | Action       | after |
|---|--------|------------|---------------------|--------------|-------|
| 4 | 37     | 7          | 4                   | output[3]=37 | 3     |
| 3 | 54     | 4          | 2                   | output[1]=54 | 1     |
| 2 | 55     | 5          | 3                   | output[2]=55 | 2     |
| 1 | 19     | 9          | 5                   | output[4]=19 | 4     |
| 0 | 91     | 1          | 1                   | output[0]=91 | 0     |

**output = [91, 54, 55, 37, 19]** — matches the hand-worked pass 1 result above. Pass 2 (exp=10, tens digit) repeats the identical procedure on this array to produce the final sorted result.

### 3.7.5 Time Complexity

- **All cases:**  $O(d(n + k))$ , where  $d$  = number of digits in the largest number,  $k$  = digit range (always 10, a constant).

**Intuitive proof:** Each of the  $d$  digit-passes is one full counting sort with a fixed range of  $k = 10$ , costing  $O(n + 10) = O(n)$  per pass. There are  $d$  such passes, one per digit position, giving:

$$T(n) = O(d \cdot (n + k)) = O(d \cdot n) \quad (\text{since } k = 10 \text{ is constant})$$

Since  $d \approx \log_{10}(M)$  where  $M$  is the maximum value, this can also be written as  $O(n \log M)$  — the value range  $M$  enters only **logarithmically**, unlike plain counting sort where it enters linearly as  $O(n + M)$ . This is precisely why radix sort remains efficient even for large-value data where plain counting sort would not.

## 4 Summary Table

| Algorithm      | Best Case          | Average Case       | Worst Case         | Space       | Stable | In-Place | Comparison-Based |
|----------------|--------------------|--------------------|--------------------|-------------|--------|----------|------------------|
| Linear Search  | $O(1)$             | $O(n)$             | $O(n)$             | $O(1)$      | —      | —        | —                |
| Binary Search  | $O(1)$             | $\Theta(\log n)$   | $\Theta(\log n)$   | $O(1)$      | —      | —        | —                |
| Bubble Sort    | $O(n)^*$           | $O(n^2)$           | $O(n^2)$           | $O(1)$      | Yes    | Yes      | Yes              |
| Selection Sort | $O(n^2)$           | $O(n^2)$           | $O(n^2)$           | $O(1)$      | No     | Yes      | Yes              |
| Insertion Sort | $O(n)$             | $O(n^2)$           | $O(n^2)$           | $O(1)$      | Yes    | Yes      | Yes              |
| Merge Sort     | $\Theta(n \log n)$ | $\Theta(n \log n)$ | $\Theta(n \log n)$ | $O(n)$      | Yes    | No       | Yes              |
| Quick Sort     | $\Theta(n \log n)$ | $\Theta(n \log n)$ | $O(n^2)$           | $O(\log n)$ | No     | Yes      | Yes              |
| Counting Sort  | $O(n+k)$           | $O(n+k)$           | $O(n+k)$           | $O(n+k)$    | Yes    | No       | No               |
| Radix Sort     | $O(d(n+k))$        | $O(d(n+k))$        | $O(d(n+k))$        | $O(n+k)$    | Yes    | No       | No               |

\*Bubble sort's best case of  $O(n)$  requires the early-exit swapped flag optimization; without it, bubble sort is always  $O(n^2)$ .

### Notes on requirements:

- **Binary search** requires the input array to already be sorted.
- **Counting sort** requires a known, reasonably small value range  $k$  (ideally  $k = O(n)$ ); otherwise its  $O(n + k)$  bound becomes impractical.
- **Radix sort** requires values with a fixed, comparable number of digits (or an equivalent structured key), and relies on the stability of its per-digit counting sort passes.
- **Any comparison-based sort** (all sorts above except counting and radix) is fundamentally limited by the comparison-sort lower bound:  $\Omega(n \log n)$  in the worst case, since  $n$  elements have  $n!$  possible orderings and each comparison yields at most one bit of information, requiring at least  $\log_2(n!) \approx n \log n$  comparisons to determine the correct ordering.